

ESTUDIO — Carpeta services/

Material de estudio para el final del TFC Bajoneá. Basado en el código real de `backend/src/main/java/com/bajonea/backend/services/` — **18 services**.

Esta es la carpeta más importante del proyecto. Aquí vive toda la lógica de negocio.

Qué es un Service y por qué es el corazón del sistema

El **service** es la capa donde **se piensa**. Los **controllers** solo reciben y devuelven; los **repositories** solo traen y guardan. **Todo lo que es "decidir" pasa acá**.

Un service hace cuatro cosas:

1. **Valida reglas de negocio** — "no podés confirmar un pedido con el carrito vacío".
2. **Orquesta repositories** — llama a varios para armar una operación completa.
3. **Convierte entidades en DTOs** — el famoso "mapeo", hecho a mano en cada service.
4. **Coordina con otros services** — por ejemplo, `PedidoService` le pide a `NotificacionService` que avise al comercio.

Las anotaciones que se repiten

Anotación	Qué hace
<code>@Service</code>	Le dice a Spring: "creá un objeto de esta clase y tenelo listo para inyectar".
<code>@RequiredArgsConstructor</code>	Lombok genera el constructor con los campos <code>final</code> , para que Spring inyecte los repositories solo.
<code>@Transactional</code>	Clave . Todo lo que hace el método es una sola unidad: o se guarda todo, o no se guarda nada.

@Transactional explicado bien — pregunta segura de mesa

Imaginate confirmar un pedido: se crea el `Pedido`, se crean 3 `DetallePedido`, se vacía el carrito y se crea una `Notificacion`. **Son 6 operaciones sobre la base**.

Si la número 4 falla, sin `@Transactional` quedarías con un pedido a medio armar: existe la cabecera pero le faltan productos, y el carrito sigue lleno. **Un desastre de datos**.

Con `@Transactional`, Spring abre una transacción al entrar al método y:

- Si termina bien → **commit**: se guarda todo junto.
- Si sale una excepción → **rollback**: se deshace todo, como si nunca hubiera pasado.

"**Todo o nada**" — es lo que en bases de datos se llama **atomicidad**, la A de ACID.

Dos patrones que se repiten en TODOS los services

Vale la pena entenderlos una vez, porque después aparecen 18 veces.

Patrón 1: el aislamiento por dueño (multi-tenancy)

```
private Producto obtenerProductoDelComercio(Integer productId, Comercio comercio) {
    Producto producto = productoRepository.findById(productId)
        .orElseThrow(() -> new RecursoNoEncontradoException("Producto no encontrado"));
    if (!producto.getComercio().getId().equals(comercio.getId())) {
        throw new RecursoNoEncontradoException("Producto no encontrado"); // <-- 404, no 403
    }
    return producto;
}
```

Lo importante: si el producto **existe pero es de otro comercio**, devuelve **404 (no encontrado)**, no 403 (prohibido).

Por qué es lo correcto, y es una respuesta excelente: un 403 le confirmaría al atacante que *"ese producto existe, pero no es tuyo"*. Probando ids uno por uno podría mapear qué productos hay en el sistema. Con 404 uniforme, **no puede distinguir "no existe" de "no es tuyo"**.

Este mismo patrón está en `ProductoService`, `PedidoService`, `NotificacionService`, `CarritoService`, `RedSocialService` y `UsuarioService`.

Patrón 2: el mapeo manual a DTO

Todos los services tienen un método privado `aResponseDTO(...)` que convierte la entidad en DTO, campo por campo, escrito a mano.

Se eligió **no usar MapStruct** (una librería que genera esos mapeos). El motivo: es **más explícito**. Mirando el método ves exactamente qué campo sale y cuál no. Con un mapper generado, agregar un campo a la entidad podría exponerlo sin querer.

GRUPO 1 — AUTENTICACIÓN Y REGISTRO

1.1 AuthService — el más grande (447 líneas)

Responsabilidad

Todo el ciclo de vida de una sesión y de las credenciales: login, logout, verificación de email, recuperación de contraseña, reactivación de cuenta, bloqueo por intentos fallidos.

No hace registro — eso es de `RegistroService` .

Sus constantes (las reglas de negocio en números)

Constante	Valor	Qué significa
MAX_INTENTOS_FALLIDOS	3	A los 3 errores de contraseña, la cuenta se bloquea.
EXPIRACION_RECUPERACION_PASSWORD_MIN	30	El código de recuperación dura 30 minutos.
EXPIRACION_REACTIVACION_CUENTA_HORAS	24	El de reactivación, 24 horas.
EXPIRACION_VERIFICACION_EMAIL_HORAS	24	El de verificación, 24 horas.
MAX_INTENTOS_TOKEN_VERIFICACION	5	A los 5 códigos errados, el token se quema.
MAX_INTENTOS_GENERACION_TOKEN	5	Cuántas veces reintenta si el código sale repetido.

La anotación de clase, que es una historia en sí misma

```
@Transactional(noRollbackFor = {
    CredencialesInvalidasException.class,
    ConflictoDeNegocioException.class })
```

El problema que resuelve (dos bugs reales, no teoría):

Bug 1 — el contador de intentos que nunca subía. El método hacía `usuario.setIntentosFallidos(n+1)` y después lanzaba `CredencialesInvalidasException` . Spring, al ver la excepción, hacía `rollback` y deshacía el incremento. Resultado: la cuenta **nunca se bloqueaba**, podías probar contraseñas infinitas.

Bug 2 — el token de verificación que nunca se quemaba. Mismo mecanismo: el contador de intentos del código se guardaba, la excepción hacía `rollback`, y el token seguía vivo. Intentos infinitos para adivinar un código de 6 dígitos.

La solución: `noRollbackFor` acotado a esas dos clases puntuales. Con esas dos excepciones no se hace `rollback`; con cualquier otra sí.

El detalle de criterio: se acotó a dos clases, **no a `RuntimeException` en general**. Si fuera genérico, ningún error haría `rollback` y cualquier fallo dejaría datos a medias.

Métodos principales

login(request, ipOrigen, userAgent)

La historia paso a paso:

- Busca el usuario **con bloqueo pesimista** (`findByEmailConBloqueo`) para que dos requests simultáneos no pisen el contador.
- Valida el estado** de la cuenta (`validarEstadoParaLogin`).
- Compara la contraseña con `passwordEncoder.matches(...)` .
- Si falla:** llama a `registrarIntentoFallido` y lanza la excepción **con los intentos restantes en el `data`** .
- Si acierta:** resetea el contador a 0, actualiza `fechaUltimoAcceso` .
- Cierra la sesión anterior** si existía (`FORZADO`) — sesión única por cuenta.
- Crea una `Sesion` nueva con `activa = true` , la IP y el navegador.
- Genera el JWT **con el `sesionId` adentro**.

Reglas clave:

- Si el email no existe → **el mismo mensaje** que si la contraseña está mal ("Email o contraseña incorrectos"). No se filtra si la cuenta existe.
- Sesión única: **entrar de nuevo cierra la sesión anterior**.

validarEstadoParaLogin(usuario) — un switch que es la regla de negocio pura

Estado	Qué pasa
ACTIVO	Pasa.
PENDIENTE	409: "Verificá tu email antes de iniciar sesión"
BLOQUEADO	409: "Cuenta bloqueada. Recuperá tu contraseña para desbloquearla"
INACTIVO	409: "Cuenta inactiva. Solicitá la reactivación de tu cuenta"
SUSPENDIDO	409: "Cuenta suspendida"

Fijate que cada mensaje **le dice al usuario qué hacer**, no solo que no puede entrar.

registrarIntentoFallido(usuario) — y el efecto dominó

```
int intentos = usuario.getIntentosFallidos() + 1;
if (intentos >= MAX_INTENTOS_FALLIDOS) {
    usuario.setEstado(EstadoUsuario.BLOQUEADO);
    cerrarSesionActivaSiExiste(usuario, TipoCierreSesion.FORZADO);
    propagarBloqueoAComercio(usuario);    // <-- lo interesante
}
```

Al bloquear la cuenta pasan **tres** cosas: se marca `BLOQUEADO` , **se cierra su sesión activa** (por si el atacante ya había entrado), y **se propaga al comercio**.

propagarBloqueoAComercio y restaurarComercioSiCorresponde — regla de negocio linda

Si el bloqueo es un **DUENO**, su comercio (si estaba `APROBADO`) pasa a `CERRADO_TEMPORALMENTE` .

Por qué: si el dueño no puede entrar a la app, no puede aceptar pedidos. Dejarlo abierto en el catálogo generaría pedidos que nadie va a atender. **El sistema lo cierra solo.**

Y la operación inversa: cuando recupera la contraseña o reactiva la cuenta, el comercio vuelve a `APROBADO` . El método `restaurarComercioSiCorresponde(usuario, estadoOrigenEsperado)` recibe **de qué estado tiene que venir**, para no reactivar por error un comercio que estaba cerrado por otro motivo.

verificarEmailConCodigo(request) vs verificarEmail(token)

Hay **dos** métodos de verificación, y la diferencia es importante:

	verificarEmail(token)	verificarEmailConCodigo(request)
Qué recibe	Solo el token	Email + código
Origen	Mecanismo viejo, por link	Mecanismo actual, código de 6 dígitos
¿Limita intentos?	No puede	Sí

Por qué el nuevo necesita el email: para limitar intentos hay que saber **contra qué token contar**. Si el único dato de entrada es el valor a adivinar, no tenés dónde llevar el contador. Pidiendo el email primero, ubicás el token del usuario y contás sus fallos.

Los 3 métodos "silenciosos" — evitar enumeración de usuarios

`solicitarRecuperacionPassword` , `solicitarReactivacionCuenta` y `reenviarVerificacion` usan:

```
usuarioRepository.findByEmail(request.getEmail()).ifPresent(usuario -> { ... });
```

Si el email no existe, no pasa nada. No hay excepción, no hay 404. El controller devuelve siempre el mismo 200 con el mensaje ambiguo.

El bug que esto corrigió: antes lanzaba `RecursoNoEncontradoException` , que llegaba al handler y devolvía un **404 real** — contradiciendo el mensaje genérico del controller. Un atacante podía distinguir emails registrados de no registrados solo mirando el status code.

Y hay un caso más fino todavía, en `solicitarReactivacionCuenta` : antes solo generaba token si la cuenta estaba `INACTIVO` . Entonces una cuenta en otro estado **no tenía token contra el cual contar intentos**, y el paso de confirmación fallaba **distinto** (sin `intentosRestantes` , sin bloqueo). Esa diferencia de comportamiento filtraba el estado real.

La solución: generar el token **siempre**, y que `confirmarReactivacionCuenta` decida si corresponde cambiar el estado:

```
if (usuario.getEstado() == EstadoUsuario.INACTIVO) {
    usuario.setEstado(EstadoUsuario.ACTIVO);
    ...
}
```

Si no estaba inactiva, **el código se consume igual** (mismo éxito visible) pero sin efecto real. Desde afuera, todos los casos se ven idénticos.

generarToken(...) — el reintento por colisión

```
for (int intento = 1; intento <= MAX_INTENTOS_GENERACION_TOKEN; intento++) {
    ...
    try { return tokenRepository.save(token); }
    catch (DataIntegrityViolationException ex) {
        if (intento == MAX_INTENTOS_GENERACION_TOKEN) throw ex;
    }
}
```

El problema: los códigos son de 6 dígitos, o sea **un millón de combinaciones**, compartidas por los 3 tipos de token, y la columna tiene un `UNIQUE` **de por vida** (las filas usadas nunca se borran). Con el tiempo, la probabilidad de que salga un código ya usado crece.

Sin el reintento, una colisión daba un **409 genérico y confuso** al usuario. Con el reintento, simplemente se genera otro código y el usuario ni se entera.

También invalida los códigos anteriores del mismo tipo antes de generar el nuevo, para que no haya varios códigos válidos a la vez.

obtenerTokenValidoPorCodigo(email, codigo, tipo) — el método más cuidado

Es el que valida los códigos de los 3 flujos. Su secuencia:

1. Busca el usuario. Si no existe → `"Código incorrecto o vencido"` (**el mismo mensaje** que si el código está mal).

- Busca su token pendiente más reciente. Si no hay → mismo mensaje.
- Si venció → "El código venció. Solicitá uno nuevo."
- Si no coincide → **suma un intento fallido**, calcula cuántos quedan, y:
 - Si quedan 0 → "Superaste el máximo de intentos permitidos."
 - Si quedan más → "Código incorrecto. Te quedan N intento(s)."

Los mensajes idénticos en los pasos 1 y 2 son deliberados: no se puede distinguir "ese email no existe" de "ese código está mal".

logout(sesionId)

```
sesion.setActiva(false);
sesion.setTipoCierre(TipoCierreSesion.MANUAL);
sesion.setFechaCierre(LocalDateTime.now());
```

Tres líneas, pero es un **logout de verdad**. Al poner `activa = false`, el `JwtAuthenticationFilter` rechaza ese token en el siguiente request, aunque su firma siga siendo válida. Un JWT puramente stateless no podría hacer esto.

1.2 RegistroService — el alta de cuentas (316 líneas)

Responsabilidad

Dar de alta clientes y comercios: crear toda la cadena de entidades, la dirección, los horarios, las redes sociales, y mandar el email de verificación.

registrarCliente(request) — la historia

- Valida email único (`existsByEmail`) → si existe, 409.
- Valida DNI único (`existsByDni`) → si existe, 409.
- Busca la localidad → si no existe, 404.
- Crea el `usuario` con la contraseña hasheada, rol `CLIENTE`, estado `PENDIENTE`.
- Crea la `Persona` (el nodo intermedio).
- Crea la `PersonaFisica` con el nombre normalizado a Title Case.
- Crea el `cliente`.
- Crea la `Direccion`, la asocia al cliente y la marca como `principal`.
- Genera el token de verificación y manda el email.

El paso 4 tiene la regla clave: nace `PENDIENTE`, no `ACTIVO`. No podés loguearte hasta verificar el email.

registrarComercio(request) — mucho más complejo

Además de todo lo anterior:

- Valida CUIT único y DNI del representante único.
- Valida los `horarios` (`validarHorarios`).
- Valida las `redes sociales` (`validarRedesSociales`).
- Valida las `modalidades de entrega` (`ComercioValidaciones`).
- Crea la cadena: `Usuario` (rol `DUENO`) → `Persona` → **`PersonaFisica`** (el representante) y **`PersonaJuridica`** (la empresa) → `Dueno` → `Comercio`.
- El comercio nace en estado `PENDIENTE` (esperando aprobación del admin).
- Guarda los horarios y las redes sociales.
- Manda el email de verificación.

Fijate que crea **DOS personas colgando de la misma `Persona`**: la física (el representante) y la jurídica (la empresa). Es exactamente el modelo de datos explicado en ESTUDIO-ENTITIES.md.

validarHorarios(horarios) — dos reglas, la segunda es la interesante

Regla 1: la hora de cierre tiene que ser posterior a la de apertura.

Regla 2: los horarios del mismo día no se pueden superponer.

```
boolean seSuperponen = actual.getHoraApertura().isBefore(otro.getHoraCierre())
    && otro.getHoraApertura().isBefore(actual.getHoraCierre());
```

Esa condición es el **algoritmo clásico de solapamiento de intervalos**, y es más elegante de lo que parece. Dos rangos `[a1, a2]` y `[b1, b2]` se solapan si y solo si `a1 < b2 && b1 < a2`. Cubre todos los casos (uno adentro del otro, cruzados, parcialmente superpuestos) con dos comparaciones.

El mensaje de error es muy bueno: *"Ya tenés un horario cargado el Miércoles de 12:00 a 15:00, que se superpone con este"* — le dice exactamente cuál es el conflicto, no un "horario inválido" genérico. Para eso está el mapa `LABELS_DIA_SEMANA`, que traduce `MIERCOLES` a `"Miércoles"`.

validarRedesSociales(redes)

```
long tiposUnicos = redesSociales.stream().map(RedSocialRequestDTO::getTipo).distinct().count();
if (tiposUnicos != redesSociales.size()) {
    throw new ValidacionException("No podés cargar dos redes sociales del mismo tipo");
}
```

Compara cuántos tipos distintos hay contra cuántas redes hay. Si no coinciden, hay repetidos. No tiene sentido cargar dos Instagram.

El token de verificación

```
.token(String.format("%06d", SECURE_RANDOM.nextInt(1_000_000)))
```

Código numérico de 6 dígitos, con `SecureRandom` (no `Random` — `SecureRandom` es criptográficamente seguro, `Random` es predecible si conocés la semilla).

Por qué 6 dígitos y no un UUID: porque el usuario lo tiene que **tippear a mano** en la pantalla de verificación. Un UUID de 36 caracteres sería impracticable.

GRUPO 2 — CATÁLOGO Y PRODUCTOS

2.1 ProductoService — 440 líneas, el segundo más grande

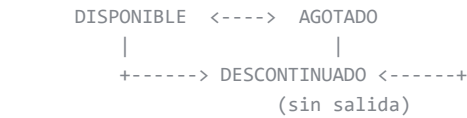
Responsabilidad

CRUD de productos del comercio, transiciones de estado, la galería de imágenes completa, y el catálogo público (que le delega `CatalogoService`).

La máquina de estados de producto

```
private static final Map<EstadoProducto, Set<EstadoProducto>> TRANSICIONES_VALIDAS = Map.of(
    EstadoProducto.DISPONIBLE,    Set.of(AGOTADO, DESCONTINUADO),
    EstadoProducto.AGOTADO,       Set.of(DISPONIBLE, DESCONTINUADO),
    EstadoProducto.DESCONTINUADO, Set.of());          // <- vacío: es terminal
);
```

Es una tabla de transiciones válidas declarada como dato, no como una cadena de `if` .



Lo importante: `DESCONTINUADO` tiene un `Set` vacío → **es un estado terminal**, no se puede salir de ahí. Si intentás, el service lanza 409: *"No se puede pasar de DESCONTINUADO a DISPONIBLE"*.

Por qué está hecho como mapa y no como `if` : agregar un estado nuevo es agregar una línea al mapa, sin tocar la lógica. Y la máquina de estados queda **legible de un vistazo**, que es la mejor documentación posible.

crearProducto(usuarioId, request)

- Resuelve el comercio del usuario (JWT → `findByDuenoId`).
- Busca la categoría → si no existe, 404.
- Crea el producto con el nombre normalizado a Title Case, **estado** `DISPONIBLE` .
- Asigna los tags.

El estado lo decide el service, no viene en el DTO — es una regla de negocio, no un dato del usuario.

editarProducto(...) — dos validaciones que valen la pena

```
if (producto.getEstado() == EstadoProducto.DESCONTINUADO) {
    throw new ConflictoDeNegocioException("No se puede editar un producto descontinuado");
}
if (imagenProductoRepository.countByProductoId(productoId) == 0) {
    throw new ConflictoDeNegocioException("Agregá al menos una foto del producto antes de guardar los cambios");
}
```

La primera es coherente con la máquina de estados: si es terminal, tampoco se edita.

La segunda es una regla de UX convertida en regla de negocio: un producto sin foto queda feo en el catálogo, así que **no se puede guardar sin al menos una imagen**. Fijate que se valida **al editar**, no al crear — porque al crear el producto todavía no existe y no se le pueden subir fotos (hace falta el id para armar la carpeta de Cloudinary).

Y sobre los tags: en la edición hace `deleteByProductoId + asignarTags` . **Borra todo y vuelve a crear**, en vez de calcular diferencias. Es más simple y no tiene casos raros; con máximo 5 tags, el costo es despreciable.

cambiarEstado(...) y el efecto en los carritos

```
if (nuevoEstado == AGOTADO || nuevoEstado == DESCONTINUADO) {
    limpiarCarritosActivos(producto);
}
```

limpiarCarritosActivos(producto) — muy buen método para contar

```
List<ItemCarrito> items = itemCarritoRepository.findByProductoId(producto.getId());
Set<Carrito> carritosAfectados = new HashSet<>();

for (ItemCarrito item : items) {
    notificacionService.crear(item.getCarrito().getCliente().getId(),
        "El producto '" + producto.getNombre() + "' ya no está disponible y fue eliminado de tu carrito.",
        TipoNotificacion.PRODUCTO_REMOVIDO_CARRITO);
    carritosAfectados.add(item.getCarrito());
}
itemCarritoRepository.deleteAll(items);

for (Carrito carrito : carritosAfectados) {
    if (itemCarritoRepository.findByCarritoId(carrito.getId()).isEmpty()) {
        carrito.setComercio(null); // <-- el bug que se corrigió acá
        carritoRepository.save(carrito);
    }
}
```

La historia: cuando un producto se agota, hay clientes que ya lo tienen en el carrito. Si lo dejás ahí, al confirmar el pedido va a fallar.

El orden importa: primero **notifica** a cada cliente, **después** borra. Si borrara primero, ya no sabría a quién avisar.

Y el bug real que se encontró acá: el último bloque no existía. Cuando un carrito quedaba **vacío** por esta limpieza, **seguía atado al comercio**. El cliente quedaba con carrito vacío pero sin poder agregar productos de otro comercio, porque el sistema creía que todavía estaba comprando en el anterior. Se corrigió reseteando `comercio = null` cuando el carrito queda sin ítems.

La galería de imágenes — 6 métodos

Método	Qué hace
generarFirmaImagen	Pide la firma a Cloudinary para una imagen nueva.
agregarImagen	Registra la imagen ya subida. Valida el límite de 5.
eliminarImagen	La borra y recalcula cuál es la principal.
reordenarImagen	Cambia el orden y recalcula la principal.
generarFirmaRecorteImagen	Firma para subir una versión recortada.
actualizarUrlImagen	Reemplaza la URL después del recorte.

recalcularImagenPrincipal(productoId) — una regla simple y robusta

```
List<ImagenProducto> imagenes = imagenProductoRepository.findByProductoIdOrderByOrdenAsc(productoId);
for (int i = 0; i < imagenes.size(); i++) {
    boolean debeSerPrincipal = (i == 0);
    ...
}
```

La regla es: la primera por orden es siempre la principal. No hay un botón de "marcar como principal" separado — se reordena y listo.

Por qué es buena: es imposible quedar en un estado inconsistente (dos principales, o ninguna). Después de cualquier operación se recalcula todo, y el resultado siempre es correcto.

Y hay una segunda validación del límite de 5: una en `CloudinaryService` (antes de firmar, para no dejar que Cloudinary reciba un archivo que se va a descartar) y otra en `agregarImagen` (al persistir). La doble validación cubre el caso de que **dos firmas se pidan casi en simultáneo**.

listarCatalogoGlobal(...) — la búsqueda del catálogo, con una decisión discutible bien explicada

```
List<Producto> productos = productoRepository.findByComercio_EstadoAndEstadoNot(APROBADO, DESCONTINUADO);
// ...filtra por categoría, por tags, por texto...
Collections.shuffle(mezclados);
// ...pagina en memoria
```

Filtra, mezcla y pagina TODO en memoria, no en SQL.

El argumento a favor (y está escrito en el código): el catálogo del MVP es chico, se trae completo y se procesa en Java. Es más simple que armar una query dinámica con `ORDER BY RAND()` .

El argumento en contra, que conviene reconocer antes de que te lo marquen: esto **no escala**. Con 10.000 productos, traerlos todos en cada request sería carísimo. La solución correcta a esa escala sería paginación en SQL (`Pageable` de Spring Data) y filtros en el `WHERE` .

Cómo contarlo: "Es una decisión consciente, documentada, apropiada para la escala actual del proyecto. Si el catálogo creciera, el cambio sería mover los filtros y la paginación a la consulta SQL." Eso muestra que entendés el trade-off, que es mejor que defenderlo como si fuera óptimo.

Un detalle del filtro de tags que sí está bien pensado: los tags combinan con **AND**, no con OR. Un producto solo pasa si tiene **todos** los tags pedidos. Por eso está el `for` que aplica un filtro por cada tag, encadenados.

listarCatalogoDelComercio(...) — el criterio de visibilidad

```
findByComercioIdAndEstadoNot(comercioId, EstadoProducto.DESCONTINUADO)
```

Excluye `DESCONTINUADO` pero NO `AGOTADO` . Es una decisión explícita: un producto agotado **se muestra igual**, marcado por su estado, para que el frontend lo **deshabilite en vez de ocultarlo**.

Por qué es mejor: si desaparece, el cliente no sabe si existió; si aparece en gris con "agotado", sabe que el comercio lo tiene y que puede volver.

2.2 CatalogoService — el más chico (43 líneas)

Responsabilidad

Ser la fachada pública del catálogo. **No tiene ninguna lógica propia** — le delega todo a `ComercioService` y `ProductoService` .

```
public List<ComercioPublicoResponseDTO> listarComerciosAprobados() {
    return comercioService.listarAprobados();
}
```

Por qué existe si no hace nada — buena pregunta

Tres razones:

1. **Semántica.** `CatalogoController` habla con `CatalogoService` . La estructura del código refleja la del dominio.
2. **Punto de composición.** `listarProductosDelComercio` sí hace algo: **primero valida que el comercio esté aprobado** (`comercioService.buscarAprobadoPorId`) y **después** trae los productos. Coordina dos services.
3. **No duplica.** El criterio de "qué es visible" ya vive en los otros dos services. Si lo reimplementara acá, tendrías dos definiciones que se pueden desincronizar.

El detalle de seguridad: usa `ComercioPublicoResponseDTO` , **nunca** `ComercioResponseDTO` . El DNI del representante no viaja sin autenticación.

2.3 CategoriaService y TagService — casi gemelos

Responsabilidad

CRUD de categorías y tags, con **baja lógica**.

Método	Qué hace	Regla de negocio
<code>crear(request)</code>	Crea una nueva.	Nombre único (<code>existsByNombre</code>) → 409 si repite. Nace con <code>activo = true</code> .
<code>editar(id, request)</code>	Cambia el nombre.	Chequea unicidad solo si el nombre cambió (ver abajo).
<code>listar()</code>	Trae todas, con su conteo de productos.	—
<code>baja(id)</code>	<code>activo = false</code> + <code>fechaBaja</code> .	No borra la fila.
<code>reactivar(id)</code>	<code>activo = true</code> + <code>fechaBaja = null</code> .	—

El detalle del `editar` que muestra cuidado

```
if (!categoria.getNombre().equals(request.getNombre())
    && categoriaRepository.existsByNombre(request.getNombre())) {
    throw new ConflictoDeNegocioException("Ya existe una categoría con ese nombre");
}
```

Sin el primer chequeo, **guardar una categoría sin cambiarle el nombre daría error** — porque `existsByNombre` encontraría... la propia categoría que estás editando. Es un bug clásico en validaciones de unicidad en edición.

El `cantidadProductos` del DTO

```
long cantidadProductos = productoRepository.countByCategoriaId(categoria.getId());
```

Es un dato **calculado**, no una columna. Sirve para que el admin vea *"Pizzas — 12 productos asociados"* antes de dar de baja algo.

Nota importante para la mesa: el sistema **no bloquea la baja de una categoría en uso**. Es una decisión de diseño: como la baja es lógica y reversible, los productos siguen apuntando a una categoría existente, solo que inactiva. Se le muestra el conteo al admin **para que decida informado**, no para prohibirle la acción.

GRUPO 3 — COMERCIO

3.1 ComercioService (217 líneas)

Responsabilidad

Perfil del comercio (ver y editar), su foto, y la validación de si acepta pedidos — que es lo más interesante.

editarPerfil(...)

- 1. Valida modalidades de entrega (al menos delivery o retiro) usando ComercioValidaciones .
- 2. Actualiza nombre (normalizado a Title Case), descripción, teléfono, email de contacto y las dos modalidades.

Solo edita el perfil público. No toca razón social, CUIT ni condición de IVA — el DTO ni siquiera los tiene.

validarAceptaPedidos(comercio) — el método clave del negocio

```
if (comercio.getEstado() != EstadoComercio.APROBADO) {
    throw new ConflictoDeNegocioException("Este comercio no está aceptando pedidos en este momento");
}
List<Horario> horarios = horarioRepository.findByComercioId(comercio.getId());
if (!estaAbiertoAhora(horarios)) {
    throw new ConflictoDeNegocioException("Este comercio está cerrado en este momento. ...");
}
```

Dos validaciones: que esté aprobado y que esté abierto en este momento.

Lo llaman CarritoService.agregarItem y PedidoService.confirmarPedido . Está en ComercioService y no duplicado en los dos, precisamente para que la regla viva en un solo lugar.

estaAbiertoAhora(horarios)

```
DiaSemana diaHoy = DiaSemana.values()[ahora.getDayOfWeek().getValue() - 1];
LocalTime horaActual = ahora.toLocalTime();

return horarios.stream()
    .filter(h -> h.getDiaSemana() == diaHoy)
    .anyMatch(h -> !horaActual.isBefore(h.getHoraApertura())
        && horaActual.isBefore(h.getHoraCierre()));
```

Cómo funciona: traduce el día de hoy al enum, filtra los horarios de ese día, y pregunta si alguno contiene la hora actual.

El -1 en el índice: DayOfWeek de Java devuelve 1 para lunes y 7 para domingo; el enum DiaSemana arranca en índice 0. Restar 1 alinea los dos.

El anyMatch es lo que hace que funcione el horario partido: si el comercio abre 12-15 y 20-00, alcanza con que la hora actual caiga en cualquiera de los dos rangos.

Y if (horarios.isEmpty()) return false; — sin horarios cargados, el comercio está cerrado. Es el criterio correcto: mejor no aceptar un pedido que no se va a poder cumplir.

Detalle honesto: el rango es [apertura, cierre) — incluye el minuto de apertura, excluye el de cierre. Y un horario que cruza la medianoche (ej. 20:00 a 02:00) no se resolvería bien con esta comparación. (A confirmar si el sistema permite cargar ese caso — la validación de "cierre posterior a apertura" de RegistroService lo impediría, así que en la práctica no debería ocurrir.)

obtenerMotivoRechazo(comercio)

```
if (comercio.getEstado() != EstadoComercio.RECHAZADO) return null;
return historialEstadoComercioRepository
    .findTopByComercioIdOrderByFechaHoraDesc(comercio.getId())
    .map(h -> ...)
```

Solo busca si el estado es RECHAZADO — evita una consulta inútil en el caso normal.

Y busca en el historial, no en el comercio, porque el modelo eliminó la columna Comercio.motivo_rechazo a favor de la tabla de historial. El motivo solo vive ahí.

Los dos mapeos separados

aResponseDTO (con representante, datos fiscales y motivo de rechazo) y aPublicoResponseDTO (sin nada de eso). Dos métodos distintos, para que sea imposible que el DTO público lleve por error un dato sensible.

3.2 RedSocialService (104 líneas)

Responsabilidad

CRUD de las redes sociales del comercio, con baja lógica.

agregar(...) — la lógica de "revivir" en vez de duplicar

```
RedSocial existente = redSocialRepository.findByComercioIdAndTipo(comercio.getId(), request.getTipo());

if (existente != null && existente.getFechaBaja() == null) {
    throw new ConflictoDeNegocioException("El comercio ya tiene una red social activa de tipo " + tipo);
}
if (activas >= MAX_REDES_SOCIALES_ACTIVAS) { // 5
    throw new ConflictoDeNegocioException("...máximo de 5 redes sociales activas");
}

if (existente == null) {
    // crea una nueva
} else {
    existente.setUrl(request.getUrl());
    existente.setFechaBaja(null); // <-- la revive
    ...
}
```

Tres reglas:

1. No dos redes activas del mismo tipo.
2. Máximo 5 activas.
3. Si diste de baja tu Instagram y lo volvés a agregar, se reutiliza la fila existente en vez de crear una nueva.

Por qué reutilizar: el `findByComercioIdAndTipo` sugiere que existe una restricción de unicidad por `(comercio, tipo)` . Crear una segunda fila del mismo tipo violaría esa restricción, aunque la primera estuviera dada de baja. Reviviéndola, el problema desaparece.

GRUPO 4 — CARRITO Y PEDIDO

4.1 CarritoService (169 líneas)

Responsabilidad

Todo el carrito: verlo, agregar, actualizar cantidad, eliminar ítems y vaciarlo.

obtenerOCrearCarrito(usuarioId) — el patrón "get-or-create"

```
return carritoRepository.findByClienteId(usuarioId)
    .orElseGet(() -> crearCarrito(usuarioId));
```

El carrito NO se crea en el registro. Se crea perezosamente, la primera vez que el cliente lo toca.

Por qué es mejor: no llenás la base de carritos vacíos de gente que se registró y nunca compró. Y como todos los métodos pasan por acá, nunca hay un caso de "carrito nulo" que manejar.

agregarItem(...) — el método con más reglas del service

La historia paso a paso:

1. Obtiene (o crea) el carrito.
2. Busca el producto → si no existe, 404.
3. ¿Está DISPONIBLE ? Si no → 409 "El producto no está disponible".
4. ¿El comercio acepta pedidos? → `comercioService.validarAceptaPedidos` (aprobado + abierto).
5. La regla del comercio único:
 - Si el carrito está vacío (`comercio == null`) → se ata a este comercio.
 - Si ya tiene otro comercio → **409**: *"El carrito ya tiene productos de otro comercio. Vacía el carrito para agregar de un comercio distinto."*
6. ¿El producto ya está en el carrito?
 - Sí → suma la cantidad al ítem existente, clampeada a 20. La nota se sobrescribe con la última.
 - No → crea un ítem nuevo.

El detalle del paso 6 — buena anécdota

```
int cantidadNueva = Math.min(itemExistente.getCantidad() + request.getCantidad(), MAX_CANTIDAD);
```

Suma en vez de rechazar, y clampea en vez de fallar. Si tenés 15 y agregás 10, quedan 20 (el máximo), no un error.

Este caso no estaba en los requisitos funcionales — era un vacío real de especificación. Se resolvió con este criterio y quedó documentado. Es un buen ejemplo de decisión de diseño tomada ante una especificación incompleta, que es una situación real de todo proyecto.

Por qué sumar y no rechazar: desde la interfaz, agregar el mismo producto dos veces es natural ("quiero dos pizzas más"). Rechazarlo con un error sería una experiencia mala por una razón que al usuario no le importa.

eliminarItem(...) y vaciarCarrito(...) — el reseteo del comercio

Ambos hacen:

```
if (itemCarritoRepository.findByCarritoId(carrito.getId()).isEmpty()) {
    carrito.setComercio(null);
    carritoRepository.save(carrito);
}
```

Cuando el carrito queda vacío, se desata del comercio. Es lo que te permite empezar a comprar en otro lado sin tener que hacer nada especial. Es la misma corrección que se aplicó en `ProductoService.limpiarCarritosActivos` .

El cálculo del subtotal

```
BigDecimal subtotal = item.getProducto().getPrecio().multiply(BigDecimal.valueOf(item.getCantidad()));
```

El precio sale del producto en el momento, y el subtotal lo calcula el backend. **Nunca se le confía el cálculo al frontend** — si el navegador mandara los precios, alguien podría manipularlos desde las herramientas de desarrollo.

Nota conceptual: el carrito **no congela precios** (a diferencia del pedido). Si el comercio sube el precio mientras tenés algo en el carrito, ves el precio nuevo. Es lo correcto: el carrito no es un compromiso, el pedido sí.

4.2 PedidoService (286 líneas) — el flujo más importante del sistema

Responsabilidad

Convertir un carrito en un pedido, y manejar la aceptación/rechazo por parte del comercio.

confirmarPedido(usuarioId, request) — la historia completa

Fase 1 — validar que se pueda:

- Busca el cliente → 404 si no existe.
- Busca el carrito → **409 "El carrito está vacío"** si no hay.
- Trae los ítems → **409 "El carrito está vacío"** si la lista está vacía.
- ¿El comercio acepta pedidos? → `validarAceptaPedidos` (aprobado + abierto).
- ¿La modalidad es compatible?
 - `DOMICILIO` pero el comercio no hace delivery → 409.
 - `RETIRO` pero el comercio no permite retiro → 409.
- Si es `DOMICILIO` :
 - Sin `direccionId` → **409** ("La dirección es obligatoria para entrega a domicilio").
 - La dirección tiene que existir, **ser del cliente que pide y no estar eliminada**. Si no → **404**.

Fase 2 — calcular:

- Valida el tope de monto por ítem (`SUBTOTAL_MAXIMO` = 99.999.999).
- Suma el subtotal de todos los ítems.
- Los cargos de servicio son **cero** (ver abajo).
- Valida que el total tampoco supere el tope.

Fase 3 — persistir:

- Crea el `Pedido` con estado `PENDIENTE` y `pagoEstado` `PENDIENTE` .
- Crea un `DetallePedido` por cada ítem, copiando el precio (`precioUnitario`).
- Vacía el carrito.
- Notifica al comercio: "Nuevo pedido recibido de Juan Pérez."

Los tres puntos de esta historia que hay que saber contar

- El paso 12 es el "snapshot de precio". El precio se copia del producto al detalle. A partir de ahí queda congelado (el campo tiene `updatable` = `false`). Si el comercio sube el precio mañana, tu pedido de hoy sigue mostrando lo que pagaste. **Un pedido es un documento histórico**.
- El paso 6 es aislamiento de datos. La validación es triple: que la dirección exista, que sea **tuya**, y que no esté eliminada. Sin el chequeo del dueño, podrías mandar el `direccionId` de otro y hacer que te lleven el pedido a la casa de un desconocido. Y devuelve **404**, no 403 — mismo criterio de no filtrar existencia.
- Los cargos de servicio en cero.

```
BigDecimal cargoServicioCliente = BigDecimal.ZERO;
BigDecimal cargoServicioComercio = BigDecimal.ZERO;
```

Las columnas existen en el modelo, la lógica de cálculo todavía no está implementada. Pertenece al tramo de `MercadoPago` y `ConfiguracionTarifa` , que está declarado en el modelo de datos pero sin implementar en los services. *(Decilo así: la estructura está lista para cuando se implemente; hoy el total es igual al subtotal.)*

aceptarPedido(usuarioId, pedidoId)

```
if (pedido.getEstado() != EstadoPedido.PENDIENTE) {
    throw new ConflictoDeNegocioException("El pedido ya fue resuelto, no está en estado PENDIENTE");
}
pedido.setEstado(EstadoPedido.EN_PREPARACION);
// notifica al cliente
```

La validación de estado evita el doble clic: si el comercio aprieta "aceptar" dos veces, el segundo intento da 409 en vez de duplicar el efecto.

Fijate que pasa a `EN_PREPARACION` , no a un estado `ACEPTADO` . El enum `EstadoPedido` **no tiene un valor `ACEPTADO`** — el modelo saltea ese paso a propósito: un pedido aceptado **ya está en preparación**. Es un detalle que conviene tener claro, porque el endpoint se llama `/aceptar` pero el estado resultante se llama distinto.

rechazarPedido(...)

Igual, más una validación propia:

```
if (request.getMotivo() == MotivoRechazo.OTRO
    && (request.getComentario() == null || request.getComentario().isBlank())) {
    throw new ValidacionException("Ingresá un comentario para especificar el motivo del rechazo.");
}
```

Si elegís "Otro" como motivo, el comentario pasa a ser obligatorio. Tiene todo el sentido: "Otro" sin explicación no le dice nada al cliente.

Es otra validación condicional entre campos, imposible de expresar con Bean Validation.

Y el mensaje de la notificación usa la etiqueta legible del enum:

```
"...fue rechazado por el comercio. Motivo: " + request.getMotivo().getEtiqueta() + "."
```

`getEtiqueta()` devuelve "Alto volumen de pedidos" en vez de `ALTO_VOLUMEN_PEDIDOS` . Ahí se ve para qué servía ese atributo del enum.

obtenerResumenHoy(usuarioId) — con su deuda técnica documentada

```
BigDecimal totalFacturadoHoy = pedidosHoy.stream()
    .filter(p -> p.getEstado() == EstadoPedido.EN_PREPARACION)    // <-- acá está el punto
    .map(Pedido::getSubtotal)
    .reduce(BigDecimal.ZERO, BigDecimal::add);
```

Cuenta `EN_PREPARACION` como "facturado".

Por qué: en el flujo implementado hoy **no existe el estado `ENTREGADO`** , así que no hay un estado final de éxito contra el cual medir. `EN_PREPARACION` es lo más cerca que hay de "este pedido se va a cobrar".

Es una deuda técnica reconocida y documentada. Cuando se implemente la máquina de estados completa, este filtro debería pasar a `ENTREGADO` .

Lo que sí está bien: `RECHAZADO` **nunca** suma. Un pedido rechazado no factura.

GRUPO 5 — SERVICES DE SOPORTE

5.1 NotificacionService (87 líneas)

Responsabilidad

El punto único de creación de notificaciones, más el listado y el marcado de lectura.

Por qué es "punto único" — buena historia de refactor

Antes, la creación de notificaciones estaba **copiada y pegada** dentro de `PedidoService` , `ProductoService` y `AdministradorService` . Cada uno armaba el objeto `Notificacion` a mano.

El **problema**: si cambiaba un campo (por ejemplo, cuando se agregó el par `entidadTipo / entidadId`), había que acordarse de tocar los tres lugares.

Se **centralizó acá**. Ahora los tres llaman a `notificacionService.crear(...)` y no saben nada de cómo se arma la entidad.

crear(...) — con sobrecarga

```
public void crear(Integer usuarioId, String mensaje, TipoNotificacion tipo) {
    crear(usuarioId, mensaje, tipo, null, null);    // delega en la versión completa
}

public void crear(Integer usuarioId, String mensaje, TipoNotificacion tipo,
    TipoEntidadNotificacion entidadTipo, Integer entidadId) { ... }
```

Dos versiones: una corta (sin referencia a entidad) y una completa. La corta delega en la larga, así la lógica está escrita una sola vez.

Dos detalles del método:

```
String mensajeTruncado = mensaje.length() > 500 ? mensaje.substring(0, 500) : mensaje;
```

Trunca a 500 caracteres, que es el largo de la columna. Sin esto, un mensaje largo (por ejemplo, un motivo de rechazo extenso) tiraría un error de base de datos. **Prefiere truncar antes que fallar** — la notificación es informativa, vale más que llegue cortada que que no llegue.

```
.usuario(usuarioRepository.getReferenceById(usuarioId))
```

`getReferenceById` en vez de `findById` . Devuelve un **proxy** sin ir a la base. Como solo necesitás la FK para el insert, no hace falta cargar el usuario entero. Es una micro-optimización correcta.

marcarLeida(usuarioId, notificacionId)

Usa `obtenerNotificacionDelUsuario` , que aplica el patrón de aislamiento: si la notificación es de otro usuario, **404** (no 403).

contarNoLeidas(usuarioId)

Una sola línea, pero es el **método más llamado del sistema** — el polling del frontend lo consulta cada 15 segundos. Por eso devuelve solo un `long` , con un `COUNT(*)` en SQL.

5.2 CloudinaryService (108 líneas)

Responsabilidad

Generar las **firmas de subida** a Cloudinary. El backend **nunca recibe el archivo**.

firmar(folder) — el método central

```
long timestamp = System.currentTimeMillis() / 1000;
Map<String, Object> paramsToSign = new HashMap<>();
paramsToSign.put("timestamp", timestamp);
paramsToSign.put("folder", folder);
paramsToSign.put("upload_preset", UPLOAD_PRESET);

String signature = cloudinary.apiSignRequest(paramsToSign, cloudinary.config.apiSecret, ...);
```

Firma tres cosas: el momento, la **carpeta** y el preset. La firma se calcula con el `apiSecret` , que **nunca sale del servidor**.

Por qué firmar la carpeta es lo importante: la firma **solo sirve para subir a esa carpeta específica**. Si un comercio consigue una firma para `productos/5/12/` , no puede usarla para subir a `productos/9/33/` — Cloudinary rechazaría la subida porque la firma no coincide.

Eso es lo que cierra el hueco de `@ValidarUrlCloudinary` , que solo valida el dominio, no la propiedad.

Las 6 firmas y sus carpetas

Método	Carpeta	Para qué
generarFirmaImagenProducto	productos/{comercioId}/{productoId}/	Galería. Valida el límite de 5.
generarFirmaRecorteImagen	Misma carpeta	Versión recortada.
generarFirmaFotoPerfilComercio	comercios/{comercioId}/perfil/	Foto del comercio.
generarFirmaFotoPerfilUsuario	usuarios/{usuarioId}/perfil/	Foto de la persona.
generarFirmaFotoPerfilRegistro	comercios/pre-registro/	Registro de comercio.
generarFirmaFotoPerfilRegistroCliente	usuarios/pre-registro/	Registro de cliente.

Las dos últimas son especiales: durante el registro el **usuario todavía no tiene id**, así que no se puede armar una carpeta scoped. Van a una carpeta fija de pre-registro. **Por eso están protegidas por el `RateLimitFotoRegistroFilter`** (5 por minuto por IP) — son las únicas firmas públicas.

(No hace falta mover el archivo después de crear la cuenta: la URL ya guardada queda con esa carpeta en el nombre, y es solo organización interna de Cloudinary, sin efecto funcional.)

El Upload Preset — dónde están las restricciones de archivo

El preset `bajonea_imagenes_mvp` está configurado en el **dashboard de Cloudinary**, no en código:

- **Formatos permitidos:** jpg, jpeg, png, webp.
- **Tamaño máximo:** 5 MB.
- **Transformación de entrada:** ancho máximo 1200px, `crop: limit` (nunca agranda), `quality: auto` .

Por qué no está en el código: porque el **backend nunca ve el archivo**. No hay nada que una anotación de validación pueda inspeccionar. Y `max_file_size` no funciona como parámetro suelto en una subida firmada — se verificó empíricamente que Cloudinary lo excluye de su propio cálculo de firma y lo ignora en silencio. **Solo se puede aplicar vía preset.**

Es un buen ejemplo de una restricción que **no puede vivir en el backend** por la arquitectura elegida, y hay que saber decir dónde vive en su lugar.

5.3 EmailService (81 líneas)

Responsabilidad

Mandar los 3 emails transaccionales del sistema, vía la API de Resend.

Método	Qué manda	Vigencia que anuncia
enviarVerificacion	Código de verificación de cuenta	24 horas, hasta 5 intentos
enviarRecuperacionPassword	Código de recuperación	30 minutos
enviarReactivacionCuenta	Código de reactivación	24 horas

enviarTextoPlano(...) — la decisión más importante de este service

```
try {
    CreateEmailResponse respuesta = resend.emails().send(params);
    log.info("Email enviado a {} (Resend id: {})", destinatario, respuesta.getId());
} catch (ResendException | RuntimeException ex) {
    log.warn("No se pudo enviar el email a {}: {}", destinatario, ex.getMessage());
}
```

Nunca relanza la excepción. Si el envío falla, lo loguea como advertencia y sigue.

Por qué, y es una decisión de diseño muy defendible: si relanzara, el `@Transactional` de `RegistroService` haría **rollback de todo el registro** solo porque el correo no salió. El usuario perdería la cuenta recién creada por un problema del proveedor de mail.

Además, el token **ya está persistido**: el usuario puede pedir el reenvío del código más adelante sin que se invalide nada.

Cómo contarlo: *"El envío de email es un efecto secundario, no parte de la transacción de negocio. Si falla, el registro tiene que sobrevivir."*

Los mensajes

Están escritos con cuidado: dicen el código, cuánto dura, cuántos intentos tenés, y **qué hacer si vos no pediste esto**. Ese último párrafo es una buena práctica de seguridad: le avisa al usuario si alguien intentó usar su email.

Nota de alcance honesta: son de **texto plano**, sin HTML ni branding. El pulido visual quedó explícitamente fuera del alcance.

5.4 ClienteService (94 líneas)

Responsabilidad

Que el cliente vea y edite su propio perfil.

El detalle técnico interesante

```
clienteRepository.findById(usuarioId)
```

Busca el `cliente` directamente con el `usuarioId` del JWT. Eso funciona porque `cliente.id` es el mismo que `usuario.id`, gracias a la cadena `@MapsId ( Cliente → PersonaFisica → Persona → Usuario )`.

Contrastalo con `comercioService`, que necesita `comercioRepository.findById(usuarioId)` — porque `comercio.id` es **autogenerado y distinto** del id del dueño. Es una consecuencia directa del modelo de datos, y muestra por qué entender la cadena `@MapsId` importa.

editarPerfil(...)

Solo toca **3 campos** (nombre, apellido, teléfono), normalizando los nombres con `TextoUtils.aTitleCase`. Actualiza también `usuario.fechaActualizacion`.

El email, el DNI y la fecha de nacimiento son de solo lectura, y no porque el service los ignore: **el DTO ni siquiera los tiene**.

El mapeo

Trae la dirección con `direccionRepository.findByClienteId(...)` y arma un DTO anidado. Si no hay dirección, queda `null` (usa `.orElse(null)` en vez de tirar excepción).

Para qué sirve exponer la dirección: para que el checkout de delivery sepa qué `direccionId` mandar en el `PedidoRequestDTO`.

5.5 AdministradorService (217 líneas)

Responsabilidad

Los listados del panel de admin, las métricas, y la **aprobación/rechazo de comercios**.

resolverAprobacion(comercioId, administradorId, request) — el método central

- La historia:
- Busca el comercio → 404 si no existe.
 - ¿Está `PENDIENTE` ? Si no → **409** "El comercio ya fue resuelto". (*Evita el doble clic.*)
 - Si rechaza, ¿mandó motivo? Si no → **400** "El motivo es obligatorio al rechazar un comercio".
 - Busca el administrador que está resolviendo.
 - Cambia el estado a `APROBADO` o `RECHAZADO` .
 - Crea la fila de `HistoriaEstadoComercio` con: el comercio, **quién** lo resolvió, el estado de origen, el de destino, el motivo y la fecha.
 - Notifica al dueño del comercio.

Los tres puntos destacables

- El paso 6 es auditoría real. Queda registrado **quién** aprobó o rechazó, **cuándo** y **por qué**. Y como la tabla es *append-only*, ese registro no se puede alterar.
- El paso 3 es la validación condicional que Bean Validation no puede hacer: el motivo es obligatorio solo si `aprobar = false` .
- El paso 7 muestra la cadena de identidad completa en acción:

```
comercio.getDueno().getPersonaJuridica().getPersona().getUsuario().getId()
```

Para llegar del comercio al usuario que hay que notificar, se recorren **cuatro** relaciones. Eso es el precio de la normalización que se explicó en ESTUDIO-ENTITIES.md — y es un buen ejemplo concreto para mostrar que entendés la estructura.

obtenerMetricas()

```
long comerciosPendientes = comercioRepository.findByEstado(PENDIENTE).size();
return new MetricasAdminResponseDTO(
    comerciosPendientes,
    comercioRepository.count(),
    clienteRepository.count(),
    categoriaRepository.countByActivoTrue(),
    tagRepository.countByActivoTrue());
```

Detalle honesto que conviene notar: los últimos cuatro usan `count()` / `countByActivoTrue()` (un `COUNT(*)` en SQL), pero el primero hace `findByEstado(...).size()` — **trae todas las filas y las cuenta en Java**. Lo correcto sería un `countByEstado` .

(Con pocos comercios pendientes es despreciable, pero si te lo señalan, reconocelo: sería una mejora fácil.)

Los tres mapeos

`aAdminResponseDTO` es el más pesado: para cada comercio arma la dirección, los horarios, el representante y las redes sociales. **Ojo:** si el admin lista 50 comercios, se ejecutan varias consultas por cada uno. Es el problema **N+1** en la práctica. (Con la escala actual no se nota; la solución sería `JOIN FETCH` en la consulta.)

5.6 GeografiaService (45 líneas)

Responsabilidad

Listar provincias y localidades para los selectores. Sin lógica de negocio, solo mapeo.

El detalle inteligente

```
private LocalidadResponseDTO aResponseDTO(Localidad localidad, String provinciaId) {
    // provinciaId ya viene del parámetro del filtro – evita navegar la relación LAZY
    return new LocalidadResponseDTO(localidad.getId(), localidad.getNombre(), provinciaId);
}
```

Usa el `provinciaId` que ya venía en el parámetro, en vez de hacer `localidad.getProvincia().getId()` .

Por qué: navegar una relación `LAZY` fuera de una transacción activa tira `LazyInitializationException` . Y aunque estuviera en transacción, generaría **una consulta por localidad** — con 200 localidades, 200 consultas extra.

Como el `provinciaId` ya lo tenés (es el filtro que pediste), es correcto reutilizarlo. Es una optimización simple con un razonamiento sólido detrás.

5.7 UsuarioService (73 líneas)

Responsabilidad

La foto de perfil del usuario (subir, cambiar, eliminar).

validarPropioUsuario(idPath, usuarioIdAutenticado)

```
if (!idPath.equals(usuarioIdAutenticado)) {
    throw new RecursoNoEncontradoException("Usuario no encontrado");    // 404, no 403
}
```

Este es el único controller que recibe un {id} en la URL, así que necesita esta validación explícita: el id de la URL tiene que ser el mismo del token.

Y devuelve 404, no 403, con el criterio ya explicado. El comentario del código lo justifica: *"no existe en el proyecto ningún patrón de un rol actuando en nombre de otro usuario"*.

5.8 TestSupportService (47 líneas)

Responsabilidad

Devolver el token pendiente de un usuario, sin necesidad de leer un email real, para que los tests automatizados corran rápido y determinísticos.

Por qué no es un agujero de seguridad

```
@Profile("test")
```

Spring solo crea este bean si la app arranca con el perfil test . Fuera de ese perfil el bean no existe, TestController no tiene a quién inyectar, y la ruta ni siquiera se registra. Un pedido a /api/v1/test/token en producción devuelve 404 porque no hay nadie que responda.

La protección es estructural, no una lista de permisos.

Y por qué es permanente, no un parche

El envío de email funciona de verdad (Resend, probado con casilla real). Este service no está para tapar algo roto — está para que los tests no dependan de una casilla externa, que los haría lentos y frágiles.

Tabla resumen de los 18 services

Service	Líneas	Responsabilidad	Regla de negocio más importante
AuthService	447	Login, logout, tokens, bloqueo	Bloqueo a los 3 intentos + sesión única
ProductoService	440	CRUD de productos y galería	Máquina de estados + límite de 5 imágenes
RegistroService	316	Alta de cuentas	Email/DNI/CUIT únicos, cuenta nace PENDIENTE
PedidoService	286	Confirmar, aceptar, rechazar pedidos	Snapshot de precio + solo desde PENDIENTE
ComercioService	217	Perfil del comercio	validarAceptaPedidos : aprobado y abierto
AdministradorService	217	Aprobación de comercios	Motivo obligatorio al rechazar + historial
CarritoService	169	El carrito	Un solo comercio a la vez
CloudinaryService	108	Firmas de subida	La firma está atada a una carpeta
RedSocialService	104	Redes del comercio	Máximo 5, sin tipos repetidos
ClienteService	94	Perfil del cliente	Solo 3 campos editables
NotificacionService	87	Notificaciones	Punto único de creación
CategoriaService	83	CRUD de categorías	Nombre único + baja lógica
TagService	83	CRUD de tags	Nombre único + baja lógica
EmailService	81	Los 3 emails	Nunca relanza la excepción
UsuarioService	73	Foto de perfil	El id de la URL debe ser el del token
TestSupportService	47	Tokens para tests	Solo bajo perfil test
GeografiaService	45	Provincias y localidades	Evita navegar la relación LAZY
CatalogoService	43	Fachada del catálogo público	Delega, nunca duplica

Preguntas típicas de mesa

"¿Qué hace un service?" Ahí vive toda la lógica de negocio: valida reglas, orquesta repositories, convierte entidades en DTOs y coordina con otros services. El controller solo recibe y devuelve; el repository solo trae y guarda.

"¿Qué es `@Transactional` ?" Hace que todo lo que hace el método sea una unidad atómica: o se guarda todo, o no se guarda nada. En `confirmarPedido` toco 4 tablas; si falla la tercera, se deshace todo y no queda un pedido a medio armar.

"Contame una regla de negocio interesante." El carrito solo puede tener productos de un comercio a la vez. Cuando agregás el primer producto, el carrito se ata a ese comercio; si intentás uno de otro, devuelvo 409. Al vaciarlo se desata. La razón es que cada comercio prepara y entrega por su cuenta — un pedido de dos comercios sería inentregable.

"¿Por qué el precio se guarda en el detalle del pedido?" Porque un pedido es un documento histórico. Si el detalle apuntara al precio actual del producto, tu historial de compras cambiaría solo cada vez que el comercio ajusta precios. Por eso el campo tiene `updatable = false` .

"¿Qué pasa si alguien intenta ver un producto de otro comercio?" Devuelvo 404, no 403. Un 403 le confirmaría que ese producto existe pero no es suyo, y probando ids podría mapear el sistema. Con 404 no puede distinguir "no existe" de "no es tuyo".

"¿Cómo evitás la fuerza bruta en el login?" Con un contador de intentos fallidos en `usuario` . A los 3, la cuenta pasa a `BLOQUEADO` , se cierra su sesión activa y, si es un dueño, su comercio pasa a cerrado temporalmente. Se destraba recuperando la contraseña.

"Contame un bug difícil." El contador de intentos fallidos nunca subía: el método era `@Transactional` , incrementaba el contador y después lanzaba la excepción de credenciales inválidas — y Spring hacía rollback de todo, incluido el incremento. La cuenta nunca se bloqueaba. Se arregló con `@Transactional(noRollbackFor = CredencialesInvalidasException.class)` , acotado a esa clase para no perder el rollback en el resto de los errores.

"¿Cómo evitás filtrar qué emails están registrados?" Los endpoints de recuperación y reactivación usan `ifPresent` : si el email no existe, no pasa nada y el controller devuelve el mismo 200 genérico. Antes lanzaban una excepción que llegaba al handler y devolvía un 404 real, que contradecía el mensaje ambiguo.

Índice de services cubiertos en este documento

Archivo	Responsabilidad en una línea
<code>AuthService.java</code>	Login, logout, verificación, recuperación, bloqueo por intentos.
<code>RegistroService.java</code>	Alta de clientes y comercios con toda su cadena de entidades.
<code>ProductoService.java</code>	CRUD de productos, máquina de estados y galería de imágenes.
<code>CatalogoService.java</code>	Fachada delgada del catálogo público.
<code>CategoriaService.java</code>	CRUD de categorías con baja lógica.
<code>TagService.java</code>	CRUD de tags con baja lógica.
<code>ComercioService.java</code>	Perfil del comercio y validación de si acepta pedidos.
<code>RedSocialService.java</code>	Redes sociales del comercio, con reutilización de filas dadas de baja.
<code>CarritoService.java</code>	Carrito con la regla de un comercio a la vez.
<code>PedidoService.java</code>	Confirmación de pedido y resolución por el comercio.
<code>NotificacionService.java</code>	Punto único de creación de notificaciones.
<code>CloudinaryService.java</code>	Firmas de subida acotadas por carpeta.
<code>EmailService.java</code>	Los 3 emails transaccionales, sin romper la transacción si fallan.
<code>ClienteService.java</code>	Perfil propio del cliente.
<code>AdministradorService.java</code>	Aprobación de comercios, listados y métricas.
<code>GeografiaService.java</code>	Provincias y localidades para los selectores.
<code>UsuarioService.java</code>	Foto de perfil del usuario.
<code>TestSupportService.java</code>	Tokens de prueba, exclusivo del perfil <code>test</code> .